



Slides for P3793R0 — Better shifting

Document number:

P3898R0

Date:

2025-11-02

Audience:

SG6

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/better-shifting-slides.cow



IMPORTANT: This document has custom controls:

- , : go to the next slide
- , : go to previous slide

Better shifting

P3793R0

Introduction

- bit-shifting (<< and >>) is not ideal:
 - hard-to-remember precedence rules
 - undefined behavior on "overlong" shifts
 - other languages, hardware use modular shift amount (C#, Java, JS)
- "fixing" bit-shifts in core language difficult
 - at best, unspecified result + erroneous behavior
- proposed: "mathematically correct" `std::shl` and `std::shr`

Motivation

"Mathematically correct" functions eliminate special cases and UB pitfalls:

Before	After
<pre>uint32_t make_mask(int n_bits) { if (n_bits >= 32) return uint32_t(-1); return (uint32_t(1) << n_bits) - 1; } uint32_t bitset = /* ... */; bool bitset_contains(size_t i) { if (i >= 32) return false; return (bitset >> i) & 1; }</pre>	<pre>uint32_t make_mask(int n_bits) { return shl(uint32_t(1), n_bits) - 1; } uint32_t bitset = /* ... */; bool bitset_contains(size_t i) { return shr(bitset, i) & 1; }</pre>

Design choices

- `std::shl` or `std::shr` N-bit shift acts like N single-bit shifts
- no wrapping behavior because it's not useful
- no separate logical/arithmetic right-shift (change signedness to choose)
- negative shifts have erroneous behavior with impl-defined result
 - no added runtime cost on release builds, no UB 🎉
- function names inspired by `std::rotl` and `std::rotr`
- shift amount passed as `int` (like `std::rotl` and `std::rotr`)
- SIMD support added, similar to `std::simd::rotl`

Possible implementation

```
template<signed-or-unsigned T>
T shl(T x, int s) noexcept {
    return unsigned(s) >= width-v<T> ? T(0)
                                       : x << unsigned(s);
}

template<signed-or-unsigned T>
T shr(T x, int s) noexcept {
    return unsigned(s) >= width-v<T> ? T(x < 0 ? -1 : 0)
                                       : x >> unsigned(s);
}
```

References

[P3724R1] *Jan Schultke*. Integer division (2025-09-29)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3724r1.html>

[GitHub] *Jan Schultke*. integer-division GitHub repository

<https://github.com/Eisenwave/integer-division>

[StackOverflowCeil] Fast ceiling of an integer division in C / C++

<https://stackoverflow.com/q/2745074/5740428>